

SKILL WEEK-4

Session 7

Task1: Process Synchronization and Child Process Monitoring Using waitpid()

Output:

```
sathwika@123:~$  
sathwika@123:~$ nano session7_task1.c  
sathwika@123:~$ gcc session7_task1.c -o session7_task1  
sathwika@123:~$ ./session7_task1  
Session 7 - Process Synchronization  
Parent PID: 506  
Parent is monitoring both child processes...  
Child 1 started. PID: 507  
Child 2 started. PID: 508  
Child 2 completed.  
Child 1 completed.  
Parent: Child 1 (PID 507) exited with status 10.  
Parent: Child 2 (PID 508) exited with status 20.  
Parent: All child processes completed.  
sathwika@123:~$
```

Observation: I learned how to synchronize and monitor child processes using waitpid(). I understood how a parent process can wait for specific child processes to complete and check their exit status using WIFEXITED() and WEXITSTATUS(). I also learned how process synchronization ensures that the parent correctly monitors the execution and completion of its child processes.

Task2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

Output:

```
sathwika@123:~$ nano session7_task2.c  
sathwika@123:~$ gcc session7_task2.c -o session7_task2  
sathwika@123:~$ ./session7_task2  
Session 7 - PATH Command Resolution  
Enter command to locate: ls  
Executable found!  
Command : ls  
Full path: /usr/bin/ls  
Executable permission: Yes  
sathwika@123:~$
```

Observation: I learned how to retrieve and use the PATH environment variable to search for executable programs. I understood how to split the PATH into individual directories and construct possible executable paths. I also learned how to check whether a command exists and has execute permission using access(), and how to handle commands that are not found in the PATH.

Session 8

Task1: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Output:

```
sathwika@123:~$ nano session8_task1.c
sathwika@123:~$ gcc session8_task1.c -o session8_task1
sathwika@123:~$ ./session8_task1
Session 8 - Variable Expansion
Enter text containing variables: Example: $USER
$USER
Original input: $USER
Expanded output: sathwika
Expansion validation:
Variable expansion completed successfully.
sathwika@123:~$
```

Observation: I learned how to detect variable references such as \$USER and \$HOME and expand them using environment variable values. I also learned how to handle undefined variables, update tokens after expansion, and support variable formats such as \$VARIABLE and \${VARIABLE}.

Task2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Output:

```
sathwika@123:~$ nano session8_task2.c
sathwika@123:~$ gcc session8_task2.c -o session8_task2
sathwika@123:~$ ./session8_task2
Session 8 - Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> echo "Hello World"

Tokens before variable expansion:
Token 1: [echo]
Token 2: [Hello World]

Variable expansion:
Token 1: [echo]
Token 2: [Hello World]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
sathwika@123:~$
```

Observation: I learned how to identify built-in commands and use a dispatch table to connect commands with their corresponding functions. I learned how built-in commands such as help, pwd, cd, and exit execute within the shell process. I also learned how to handle invalid commands and maintain shell state when commands such as cd change the current directory.